

7/10 미팅

LLM-based Multi-Agent Systems에서의 Latent Communication 개요

하서준

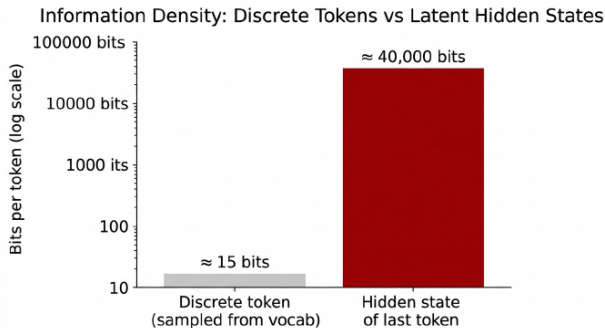
DGIST

2026년 7월 10일

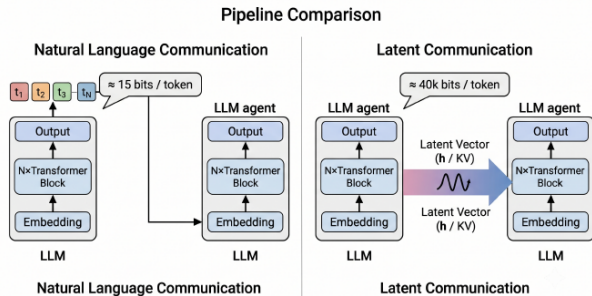
Latent Communication · Hidden States · KV-Cache · Multi-Agent LLMs

왜 latent communication이 필요한가?

기존 LLM-based MAS의 기본 구조



(a) Information density: ≈ 15 bits per token.



(b) Pipeline comparison: NL-Comm vs. Latent-Comm.

Text-only communication의 세 가지 한계

1. 높은 inference cost

모든 메시지마다 sender는 token을 생성하고, receiver는 그 token을 다시 encoding해야 한다.

2. Discretization 과정의 정보 손실

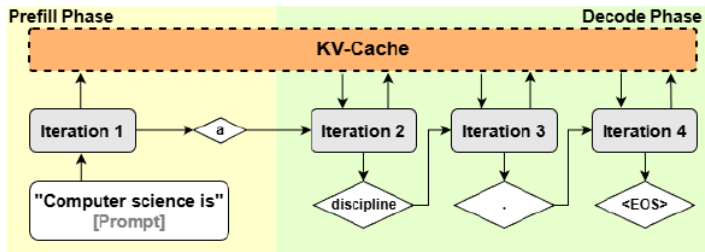
고차원 hidden state가 vocabulary token 하나로 압축된다. 논문은 token의 정보량을 대략 15–17 bits로 보고, hidden state는 훨씬 큰 capacity를 가진다고 설명한다.

3. 자연어의 중복성과 모호성

자연어는 사람이 읽기 좋지만 상황하고, hedge 표현이나 모호한 지시어 때문에 task-relevant density가 낮을 수 있다.

Prefill phase와 Decode phase

LLM inference는 크게 두 단계로 나뉜다.



Prefill phase

- ▶ 입력 prompt 전체를 한 번에 Transformer에 통과시킨다.
- ▶ 모든 input token의 hidden state와 KV-cache가 만들어진다.
- ▶ prompt가 길수록 비용이 크게 증가한다.

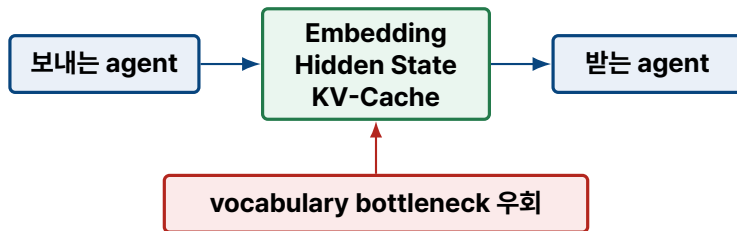
Decode phase

- ▶ 만들어진 KV-cache를 사용해 다음 token을 하나씩 생성한다.
- ▶ 각 step에서는 직전 token과 누적 KV-cache를 참조한다.
- ▶ 생성 길이에 비례해 반복적으로 비용이 든다.

Prefill에서는 모든 input token의 hidden state와 KV-cache가 생성되고, decode에서는 최신 token의 hidden state와 계속 커지는 KV-cache가 핵심 latent가 된다.

무엇이 달라질 수 있나?

핵심 아이디어: agent가 자연어를 생성하지 않고 내부 연속 표현을 직접 전달한다.



이 논문의 3-axis 분석 틀

WHAT
무엇을 보낼까?
Embedding / Hidden State / KV-Cache

WHICH
어디에 맞출까?
layer / latent alignment

HOW
어떻게 합칠까?
concatenate / prepend / fuse

방법 = (WHAT, WHICH, HOW)
 정보 종류 alignment fusion

자연어 vs. Latent communication: 언제 무엇을 쓰나?

Latent communication이 natural language를 완전히 대체하는 것은 아니다.

자연어 Communication

자연어를 사용해야 하는 경우

- ▶ 사람의 해석이 필요할 때
- ▶ 자연어 message는 사람이 즉시 읽을 수 있음
- ▶ debugging, alignment auditing, safety review에 유리
- ▶ human-AI interaction에서 그대로 설명하고 검토 가능

사용할 때

- ▶ final answer를 제공할 때
- ▶ 사용자에게 justification을 보여줘야 할 때
- ▶ human oversight / cross-organization interoperability 필요

Latent Communication

Latent를 선호하는 경우

- ▶ 강하게 연결된 **agents**
planner → executor처럼 바로 이어지는 agents
- ▶ 중간 **communication**
사용자가 message를 볼 필요가 없는 단계
- ▶ 공유되거나 정렬된 **latent space**
same backbone 또는 compatible architectures
- ▶ **latency**가 핵심 제약인 경우
real-time pipeline, edge deployment, large agent counts

WHICH: 송신자와 수신자를 어디서 맞출까?

1. Space: latent space를 맞춤

같은 model은 거의 바로 공유 가능하다. 같은 backbone의 fine-tune은 projection head로 맞출 수 있고, heterogeneous model은 learned alignment가 필요하다.

2. Layer: 어느 layer를 연결할지 정함

대표적으로 **Last** → **First**, **All** → **Corresponding**, **Selected** → **Selected**가 있다.

3. Select: 필요한 일부만 고름

모든 layer를 넘기면 정보는 많지만 비용이 크다. task와 receiver에 중요한 layer만 고르면 latency와 memory를 줄일 수 있다.

즉 WHICH는 공간 정렬(Space), 레이어 매핑(Layer), 선택적 전달(Select)의 문제로 볼 수 있다.

HOW: 수신자의 computation에 어떻게 합칠까?

1. **Concat: latent token**을 뒤에 붙임

Embedding/hidden state를 receiver sequence axis에 추가한다. 가장 단순하지만 sequence length가 늘어난다.

2. **Prepend: KV-cache** 앞에 붙임

Sender KV를 receiver cache 앞에 두어 과거 memory처럼 attention하게 한다. KV-cache 계열의 기본 방식이다.

3. **Fuse: projection** 후 섞음

Add, gate, cross-attention 등으로 sender latent를 receiver computation에 섞는다. 가장 유연하지만 alignment와 training이 필요하다.

직관적으로는 붙이기(Concat), 기억으로 앞에 두기(Prepend), 계산해서 섞기(Fuse)의 세 가지로 볼 수 있다.

실험적 관찰: 무엇이 잘 먹히나?

1. Long context 일수록 이득이 큼

receiver가 긴 prompt를 다시 encoding하지 않아도 되기 때문.

2. Same-model agents가 지배적

대부분 sender와 receiver가 같은 backbone을 공유한다고 가정.

3. KV-cache가 emerging default

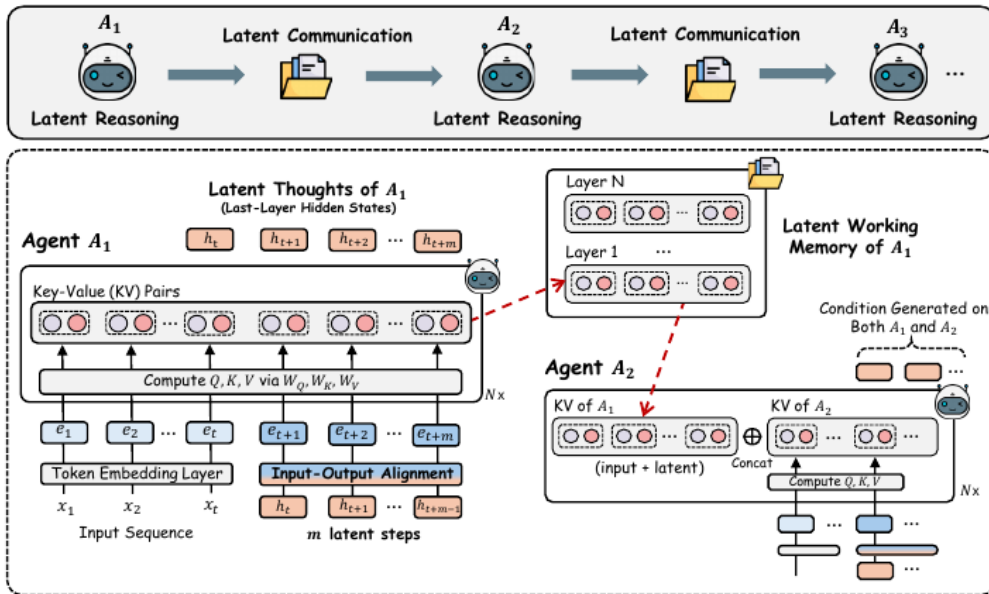
18개 방법 중 9개가 KV-cache를 communicated quantity로 사용.

핵심 trade-off: 정보량이 많을수록 latency는 줄기 쉽지만, payload와 architecture dependence는 커진다.

Latent Collaboration in Multi-Agent Systems

ICML, 2026

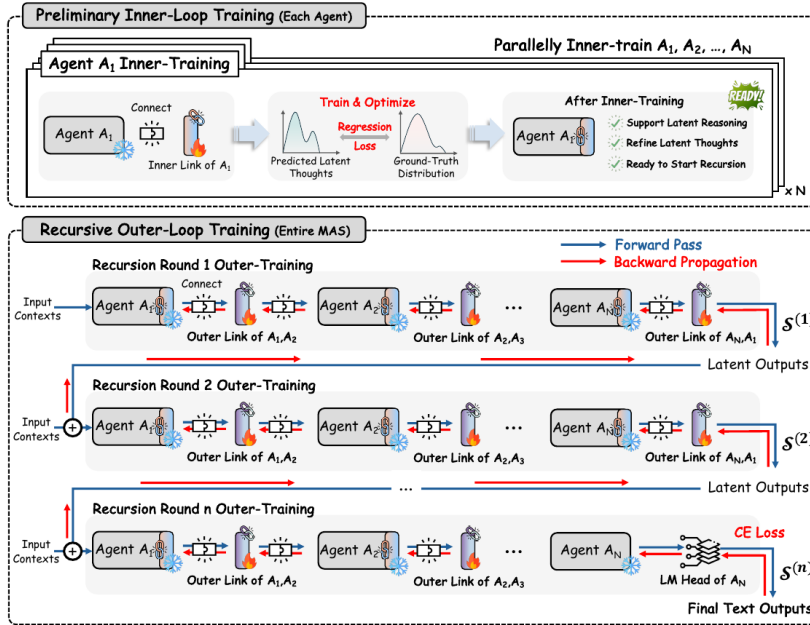
Latent Collaboration (LatentMAS)



자연어 communication은 느리고 hidden reasoning을 버린다.
 ⇒ **latent thought**와 **KV-cache**를 다음 agent의 **memory**로 직접 넘기자.

Recursive Multi-Agent Systems

arXiv, 2026



MAS를 text message들의 왕복으로 보면 느리고 학습이 끊긴다.
 ⇒ agent들을 latent recursion loop로 묶고 RecursiveLink만 학습하자.

Cache-to-Cache: Direct Semantic Communication Between Large Language Models

ICLR, 2026

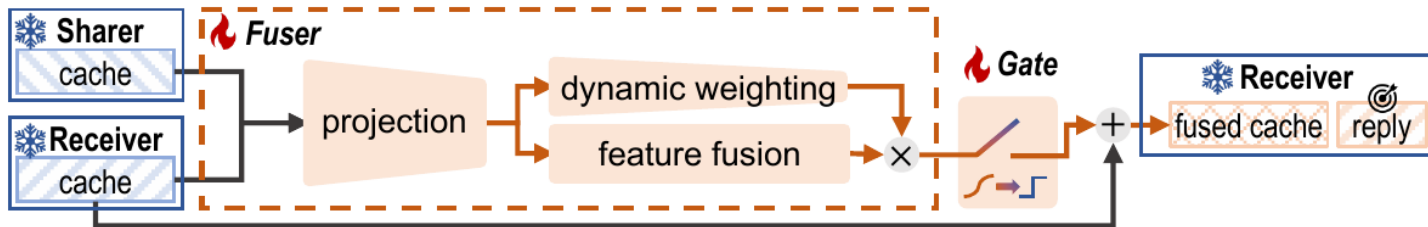


Figure 5: Cache fuser architecture and training scheme. *Fuser* projects and combines KV-Caches, then adds the result to Receiver’s cache through a learnable gate. LLMs are frozen during training.

기존 전달 방식은 sharer 정보에 집중하고 receiver의 현재 맥락은 충분히 보지 않는다.
⇒ sharer/receiver KV를 함께 보고, 필요한 비율만 receiver cache에 주입하자.

Related works: 핵심 아이디어

논문/계열

기존 문제와 제안

LRAgent

ICML'26

Multi-LoRA agent는 backbone은 공유해도 같은 trajectory의 KV-cache를 agent 별로 따로 만든다.

⇒ **base KV**는 공유하고 **LoRA** 차이는 **low-rank cache**로만 저장하자.

KVComm

NeurIPS'25

Multi-agent workflow에서 같은 context가 반복되지만 agent별 prefix 때문에 KV-cache offset이 달라진다.

⇒ **overlap context**의 KV-cache를 **position alignment**해서 재사용하자.

DroidSpeak

NSDI'26

Full cache를 공유하면 정확도가 깨지고, 전부 recompute하면 느리다.

⇒ **민감한 layer**만 recompute하고 나머지 **layer cache**는 공유하자.

MoT

arXiv'25

Text message만으로는 세밀한 internal reasoning을 주고받기 어렵다.

⇒ **agent hidden state**를 **interaction layer**에서 **cross-attention/fusion**으로 섞자.

Coconut

COLM'25

CoT는 reasoning을 전부 token으로 써야 해서 길고 discrete token에 묶인다.

⇒ **중간 reasoning**을 **continuous latent state**로 굴리자.

공통 흐름: communication을 자연어 밖으로 옮기고, KV-cache / hidden state / latent thought를 직접 공유하거나 변환한다.

우리 태스크: sequential clinical decision making

목표: 제한된 관찰 정보에서 진단을 맞히되, 필요한 검사만 순차적으로 추가한다.



세 가지 판단 요소

1

어떤 질환인가?

disease classification

2

언제 충분한가?

stop / continue judgment

3

어떤 검사를 추가할까?

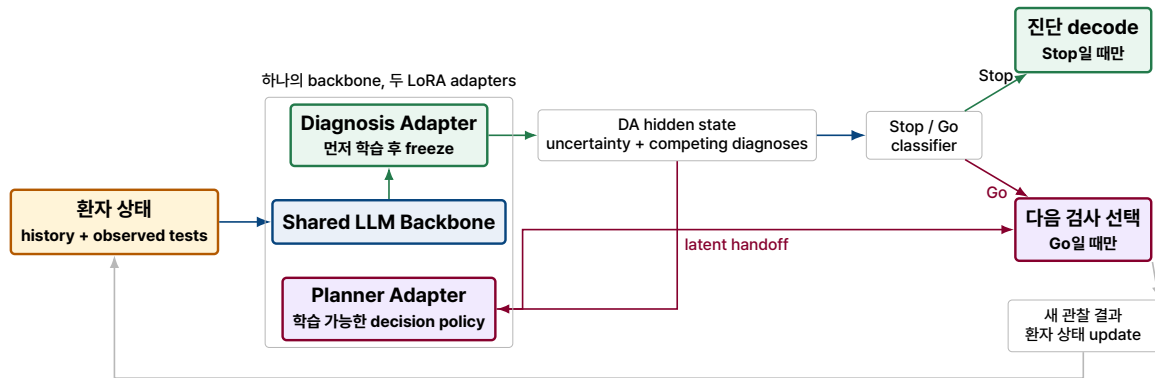
next test selection

Latent 관점에서의 비효율

1. 질환별 **uncertainty**와 후보 간 상대 구조의 이산화
연속적인 hidden state 안의 후보 질환 분포와 A vs. B의 미세한 차이가 자연어 token으로 압축된다.
2. 공통 환자 정보의 반복 입력
각 agent는 prefix만 다르고, 같은 환자 history와 observed tests를 매 step 다시 prefill한다.
3. 중간 판단마다 불필요한 **text decoding** 발생
사용자가 볼 필요 없는 내부 decision까지 자연어로 생성하면서 decode 비용이 누적된다.

제안 방향: selective latent handoff for CDM

목표: 하나의 backbone 안에서 diagnosis와 planning을 분리하되, 중간 정보는 token이 아니라 latent로 전달한다.



Re-encoding 회피

동일 환자 정보를 매 step마다 text로 다시 prefill하지 않는 방향.

Token bottleneck 완화

label 하나 대신 uncertainty와 competing diagnoses를 hidden state로 전달.

Selective decoding

중간 단계는 latent, 최종 진단 또는 설명이 필요할 때만 자연어 decode.